

Module 3

OOPC-JAVA-EVENT HANDLING

PREPARED BY: MRS. SANGEETHA M S, ASST. PROFESSOR, SCSEM-MANGALORE



Event Handling

Java supports **event-driven programming**, which means your program responds to **events**—things that happen when the user interacts with the computer.

An **event** is an action performed by the user, such as:

Clicking a button

Moving the mouse

Typing on the keyboard

Selecting a checkbox

Whenever one of these actions happens, **Java generates an event**.

Any program with a **Graphical User Interface (GUI)**—like a window with buttons, menus, or text fields—must respond to user actions.

Event Listeners:

A listener is just a **special method** that runs when an event occurs.
To respond to an event, your code must use a **listener**.

Example:

ActionListener → handles button clicks

MouseListener → handles mouse actions

The Delegation Event Model

- In this model, one object generates an event, and another object handles it.
- The object that generates the event is called the **source**, and the object that wants to react to that event is called the **listener**.
- Whenever something happens—like
a button being clicked,
a key being pressed, or
the mouse moving— the source creates an event object.
- This event object carries information about what happened.
- But the source does not handle the event itself. Instead, it “delegates” the handling task to a listener. That is why this approach is called the ***delegation event model***.

Source

Event

Listener

Examples:

A button is a source of **ActionEvent**

A window is a source of **WindowEvent**

Keyboard input generates **KeyEvent**

- Register with the source
- Wait for the event
- When the event occurs, they run a method to handle it

To **register** a listener: `button.addActionListener(myListener);`

To **remove** a listener: `button.removeActionListener(myListener);`

Some components allow **multiple listeners** → **multicasting**

Some allow only **one listener** → **unicasting**

Event Classes

Java's **event handling mechanism** is based on the *delegation event model*. This model uses **event classes** and **listener interfaces** to handle different kinds of user actions—like mouse clicks, key presses, window closing, scrolling, etc.

In Java's GUI programming (AWT and Swing), **user interactions**—such as clicking a button, typing on the keyboard, moving the mouse, resizing a window—are handled using a system called the **delegation event model**.

At the center of this model are the **event classes**, which describe different types of events.

What is AWT in Java?

AWT stands for **Abstract Window Toolkit**.

It is the **original** Java GUI (Graphical User Interface) toolkit, introduced in JDK 1.0.

Key Features of AWT

- Uses **native system resources** (platform-dependent)
 - AWT components look different on Windows, Linux, Mac because they use OS-specific code.
- Provides basic GUI components like:
 - Button
 - Label
 - TextField
 - Checkbox
 - Frame
 - Menu

Advantages of AWT

- Fast and lightweight because it uses native OS components.
- Good for very simple GUI applications.

Disadvantages of AWT

- Limited number of components
- **Platform-dependent** (not fully consistent across systems)
- Less powerful compared to Swing

What is Swing in Java?

Swing is an advanced GUI toolkit built on top of AWT.

Introduced in Java 1.2 as part of **Java Foundation Classes (JFC)**.

Key Features of Swing

•Platform-independent

It does *not* use native OS components; it draws its own components.

•Rich set of GUI controls:

- JButton
- JTextField
- JTable
- JTree
- JComboBox
- JTabbedPane
- JOptionPane

...and many more.

•Supports **pluggable look and feel** (can change the UI style)

•More powerful and flexible than AWT

Advantages of Swing

- More components (tables, trees, sliders, tab panes)
- Consistent appearance on all platforms
- Customizable and highly extensible
- Pure Java (platform independent)

Disadvantages of Swing

- Slightly slower than AWT because it doesn't use native OS widgets

1. **EventObject** – Root of All Event Classes

- The **EventObject** class (in `java.util`) is the **topmost superclass** for *every* event in Java.
- No matter what the event is—mouse click, key press, window closing—it ultimately inherits from **EventObject**.

Constructor:

`EventObject (Object src)`

- src** = the object that generated the event
e.g., a JButton clicked by the user.

Method	Purpose
<code>getSource()</code>	Returns the object that raised the event
<code>toString()</code>	Returns a string representing the event

2. AWTEvent – Superclass of All AWT/Swing GUI Events

Located in java.awt

This class extends **EventObject**.

It is the base class for all events used by GUI components (buttons, frames, textfields, etc.).

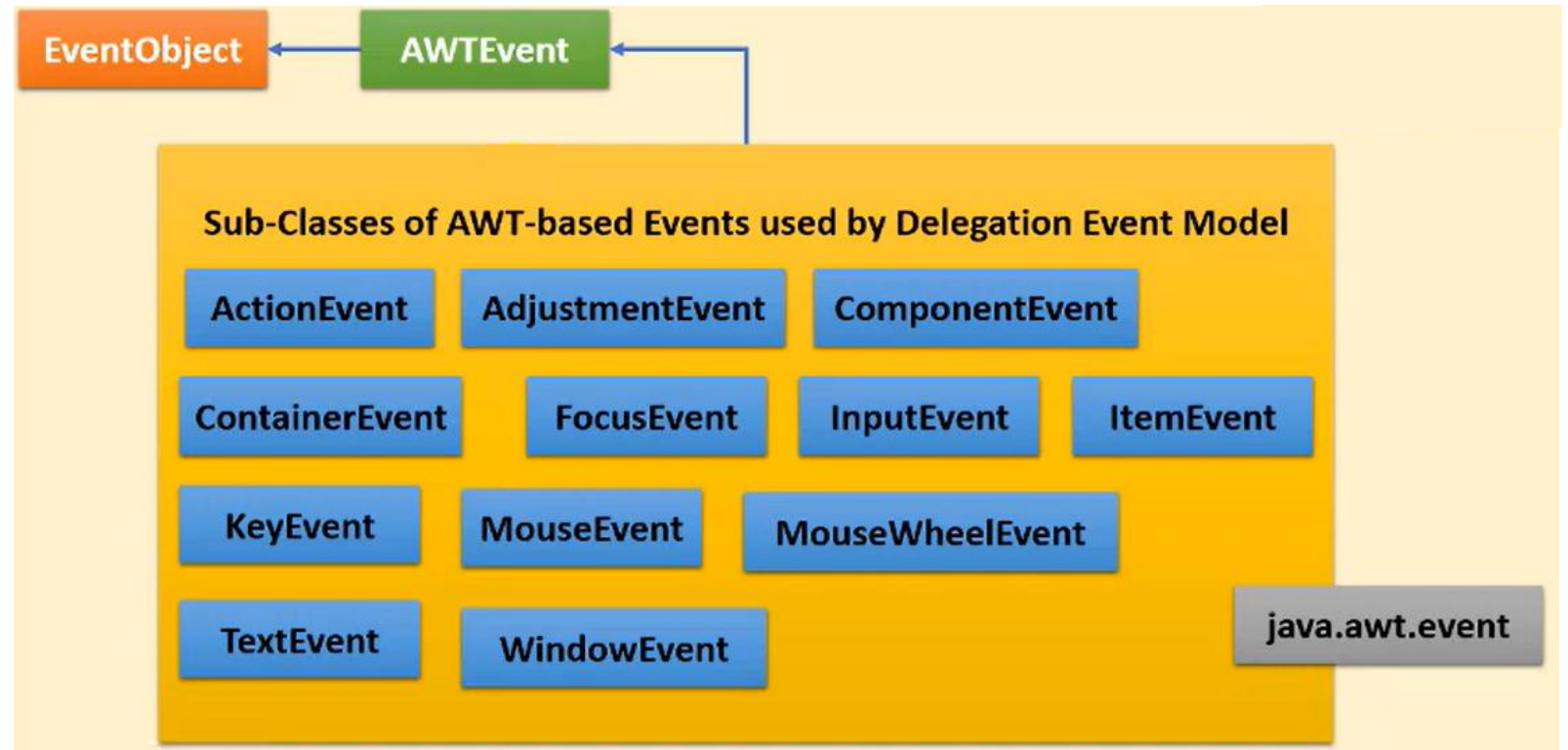
Method	Purpose
getID()	This returns the type of the event

(e.g., KEY_PRESSED, MOUSE_CLICKED, WINDOW_OPENED).

All the specific GUI events you commonly use—such as **ActionEvent**, **MouseEvent**, **KeyEvent**, etc.—are subclasses of **AWTEvent**.

Event Classes

1. The ActionEvent class
2. The AdjustmentEvent Class
3. The ComponentEvent Class
4. The ContainerEvent Class
5. The FocusEvent Class
6. The InputEvent Class
7. The ItemEvent Class
8. The KeyEvent Class
9. The MouseEvent Class
10. The MouseWheelEvent Class
11. The TextEvent Class
12. The WindowEvent Class



Event Class	Description
ActionEvent	Generated when a button is pressed, a list item is double-clicked, or a menu item is selected.
AdjustmentEvent	Generated when a scroll bar is manipulated.
ComponentEvent	Generated when a component is hidden, moved, resized, or becomes visible.
ContainerEvent	Generated when a component is added to or removed from a container.
FocusEvent	Generated when a component gains or loses keyboard focus.
InputEvent	Abstract superclass for all component input event classes.
ItemEvent	Generated when a check box or list item is clicked; also occurs when a choice selection is made or a checkable menu item is selected or deselected.
KeyEvent	Generated when input is received from the keyboard.
MouseEvent	Generated when the mouse is dragged, moved, clicked, pressed, or released; also generated when the mouse enters or exits a component.
MouseWheelEvent	Generated when the mouse wheel is moved.
TextEvent	Generated when the value of a text area or text field is changed.
WindowEvent	Generated when a window is activated, closed, deactivated, deiconified, iconified, opened, or quit.

Table: Main Event Classes in java.awt.event

1. The ActionEvent class

An ActionEvent is generated when a button is pressed, a list item is double-clicked, or a menu item is selected.

The ActionEvent class defines four integer constants that can be used to identify any modifiers associated with an action event:

ALT_MASK,
CTRL_MASK,
META_MASK (Windows key / Command key)
SHIFT_MASK.

`ActionEvent` objects are created when an action occurs. Java provides three constructors:

`ActionEvent(Object src, int type, String cmd)`

`ActionEvent(Object src, int type, String cmd, int modifiers)`

`ActionEvent(Object src, int type, String cmd, long when, int modifiers)`

- **src** → The object that generated the event (e.g., Button object)
- **type** → Type of event (e.g., `ActionEvent.ACTION_PERFORMED`)
- **cmd** → Command string (usually same as button label)
- Adds **modifiers** → keys pressed when action occurred
- Adds a **timestamp** → when the event happened

1. `getActionCommand()`

<code>String getActionCommand()</code>	Returns the command string of the event.
--	---

Example: If a button labeled "OK" is clicked,
`getActionCommand()` returns "OK".

This helps you identify *which* button or menu item was clicked.

2. `getModifiers()`

<code>int getModifiers()</code>	Returns an integer value representing modifier keys pressed at the moment of the event.
---------------------------------	--

Example:

If user clicks button while holding **CTRL + SHIFT**,
`getModifiers()` returns a value representing both keys.

3. getWhen()

long getWhen()	Returns the time when the event occurred
----------------	---

Example:

```
long time = e.getWhen();  
System.out.println("Event time: " + time);
```

2.getModifiers()

int getModifiers()	Returns an integer value representing modifier keys pressed at the moment of the event.
--------------------	--

Example:

If user clicks button while holding **CTRL + SHIFT**,
`getModifiers()` returns a value representing both keys.

2. The AdjustmentEvent Class

An AdjustmentEvent is generated by a scroll bar. It occurs when the **user moves or clicks a scroll bar**.

A scroll bar can change in different ways:

Drag the slider

Click the arrows

Click inside the scroll bar

Each of these actions generates an **AdjustmentEvent**.

BLOCK_DECREMENT	The user clicked inside the scroll bar to decrease its value.
BLOCK_INCREMENT	The user clicked inside the scroll bar to increase its value.
TRACK	The slider was dragged.
UNIT_DECREMENT	The button at the end of the scroll bar was clicked to decrease its value.
UNIT_INCREMENT	The button at the end of the scroll bar was clicked to increase its value.

Constructor of AdjustmentEvent is

AdjustmentEvent(Adjustable src, int id, int type, int data)

- src** → The object that generated the event (usually the ScrollBar)
- id** → Event ID (ADJUSTMENT_VALUE_CHANGED)
- type** → Type of adjustment (one of the 5 constants above)
- data** → The new scroll bar value

Methods are:

Adjustable getAdjustable()	Returns the scroll bar that caused the event.
int getAdjustmentType()	Returns how the scroll bar was changed
int getValue()	Returns the current value of the scroll bar after the change.

3. The ComponentEvent Class

A **ComponentEvent** occurs when something about a GUI component changes.

A *component* can be a **button, label, text field, window**, etc.

A ComponentEvent is generated when:

COMPONENT_HIDDEN – Component becomes invisible

COMPONENT_SHOWN – Component becomes visible

COMPONENT_MOVED – Component changes its position

COMPONENT_RESIZED – Component changes its size

Example:

If you resize a window → **COMPONENT_RESIZED** event is generated.

Constructor:

ComponentEvent(Component src, int type)

- **src** → the component that created the event (for example, a Button or Frame)
- **type** → which kind of event happened (resize, move, hide, show)

It is the **parent class** for many other events like:

ContainerEvent

FocusEvent

KeyEvent

MouseEvent

WindowEvent

This means all those events inherit some basic features from ComponentEvent.

Component getComponent()	It returns the component on which the event happened.
--------------------------	---

Example:

If a **Button** is resized:

- The event object stores the information
- Calling `event.getComponent()` will return **that Button**

Example:

```
public void componentResized(ComponentEvent e) {  
    Component c = e.getComponent();  
    System.out.println("Resized component: " + c.getName());  
}
```

If a Frame was resized, this will print:

Output: Resized component: frame1

4. The ContainerEvent Class

A **ContainerEvent** happens when a **component is added to or removed from** a container.

A **container** is something that can hold other components, such as:

Frame

Panel

Applet

Window

There are only two types of **Container Events** :

COMPONENT_ADDED → A component was added

COMPONENT_REMOVED → A component was removed

Example:

If you add a Button to a Panel → **COMPONENT_ADDED** event happens.

Constructor

ContainerEvent(Component src, int type, Component comp)

- src** → the container where the event happened

(example: Panel or Frame)

- type** → what happened

(added or removed)

- comp** → the component that was added or removed

(example: Button or Label)

Container getContainer()	It returns the container in which the event happened.
Component getChild()	It returns the component that was added or removed.

Example:

If a Button is added to a Panel, `getContainer()` will return **that Panel**.

If you added a Button to a Frame:

`getContainer()` → returns the Frame

`getChild()` → returns the Button

Example:

```
public void componentAdded(ContainerEvent e) {  
    Container container = e.getContainer();  
    Component child = e.getChild();  
  
    System.out.println("Event happened in: " +  
container.getName());  
    System.out.println("Component added: " + child.getName());  
}
```

If a Button is added to a Panel, the output will be:

Event happened in: panel1
Component added: button1

5. The FocusEvent Class

A **FocusEvent** happens when a component:

- **gains focus** → FOCUS_GAINED
- **loses focus** → FOCUS_LOST

A component has *focus* when it is ready to receive keyboard input.

Example:

If you click on a TextField → it **gains focus**.

If you then click on a button → TextField **loses focus**, button may gain focus.

Constructors

FocusEvent(Component src, int type)

- src = component where event happened
- type = FOCUS_GAINED or FOCUS_LOST

FocusEvent(Component src, int type, boolean temporaryFlag)

It tells whether the focus change is **temporary or permanent**.

FocusEvent(Component src, int type, boolean temporaryFlag, Component other)

This also gives the **other component** involved in the focus change.

Component getOppositeComponent()	It returns the other component involved in the focus change.
boolean isTemporary()	Returns true if the focus change is temporary. Returns false if the focus change is permanent.

Example for `getOppositeComponent()`:

1.If TextField **gains** focus and Button **loses**,

→ opposite component = Button

2.If TextField **loses** focus and Button **gains**,

→ opposite component = Button

So this method tells you **which component swapped focus**.

Example for boolean `isTemporary()`:

- You click another button → focus change is permanent → returns **false**

- You drag a scrollbar → focus change is temporary → returns **true**

6. The InputEvent Class

InputEvent is an **abstract class** in Java.

- It extends **ComponentEvent**.
- It is the **superclass** for events generated from **user input**.

Its two main subclasses are:

KeyEvent (keyboard actions)

MouseEvent (mouse actions)

Provides:

- Modifier key detection (Shift, Ctrl, Alt)
- Methods like `isShiftDown()`, `getModifiersEx()`

How to Check if a Modifier Was Pressed

InputEvent provides simple methods:

```
isAltDown()  
isControlDown()  
isShiftDown()  
isMetaDown()  
isAltGraphDown()
```

Example use:

```
if (e.isControlDown()) {  
    System.out.println("Ctrl key was pressed");  
}
```

7. The ItemEvent Class

Generated when:

- A checkbox is checked/unchecked
- A list or choice selection is changed
- A checkable menu item is selected/deselected

Common sources: Checkbox, Choice, List, CheckboxMenuItem

Listener: ItemListener

Method: itemStateChanged(ItemEvent e)

8. The KeyEvent Class

Generated when keyboard input occurs:

Key pressed

Key released

Key typed

Listener: KeyListener

Methods: keyPressed(), keyReleased(), keyTyped()

9. The MouseEvent Class

Generated when mouse is:

Clicked

Pressed

Released

Moved

Dragged

Entered or exited a component

Listener: `MouseListener` and `MouseMotionListener`

Methods: `mouseClicked()`, `mouseMoved()`, `mouseDragged()`, etc.

10. The MouseWheelEvent Class

Generated when:

Mouse wheel is rotated (scrolling)

Listener: MouseWheelListener

Method: mouseWheelMoved(MouseWheelEvent e)

11. The TextEvent Class

Generated when:

Text in a text field or text area changes

Common sources: TextField, TextArea

Listener: TextListener

Method: textValueChanged(TextEvent e)

12. The WindowEvent Class

Generated when a window is:

Opened

Closed

Activated / Deactivated

Iconified / Deiconified

Closing requested (user clicks close button)

Common source: Frame, Dialog

Listener: WindowListener

Methods: windowOpened(), windowClosing(), windowClosed(), etc.

Sources of Events

1. Button

A button generates an **ActionEvent**

Happens when the **button is clicked/pressed**

Example: Clicking "Submit" or "OK"

Listener used: `ActionListener`

2. Check Box

Generates an **ItemEvent**

Occurs when the checkbox is:

Selected (checked)

Deselected (unchecked)

Listener: `ItemListener`

Sources of Events

3. Choice

A dropdown list (like a combo box)

Generates **ItemEvent**

When the user **changes the selected option**

Listener: `ItemListener`

4. List

Generates two types of events:

ActionEvent

When an item is **double-clicked**

ItemEvent

When an item is **selected or deselected**

Listeners: `ActionListener`, `ItemListener`

Sources of Events

5. Menu Item

Generates:

ActionEvent

When a normal menu item is **selected**

ItemEvent

When a **checkable** menu item (like a toggle) is selected/deselected

Example: “View → Show Toolbar”

Listeners: ActionListener, ItemListener

6. Scroll Bar

Generates **AdjustmentEvent**

Occurs when:

Slider is moved

Value increases/decreases

User drags the scroll bar

Listener: AdjustmentListener

Sources of Events

7. Text Components

Includes TextField and TextArea

Generates **TextEvent**

When:

- Text content is **changed**

- User **types or edits** input

Listener: TextListener

8. Window

Generates **WindowEvent**

When a window is:

- Opened**

- Closed**

- Activated / Deactivated** (focus gained/lost)

- Iconified / Deiconified** (minimized/ restored)

- Closing requested** (close button clicked)

Listener: WindowListener

Event Listener Interfaces

To become a listener, a class must **implement a listener interface** from the java.awt.event package.

A **listener interface** defines **one or more methods** that must be written to handle events. When your class implements a listener interface:

You are telling Java:

“I am interested in this type of event.”

Example:

```
class MyClass implements ActionListener
{ public void actionPerformed(ActionEvent e)
  { // code to handle button click
  }
}
```

Interface	Description
ActionListener	Defines one method to receive action events.
AdjustmentListener	Defines one method to receive adjustment events.
ComponentListener	Defines four methods to recognize when a component is hidden, moved, resized, or shown.
ContainerListener	Defines two methods to recognize when a component is added to or removed from a container.
FocusListener	Defines two methods to recognize when a component gains or loses keyboard focus.
ItemListener	Defines one method to recognize when the state of an item changes.
KeyListener	Defines three methods to recognize when a key is pressed, released, or typed.
MouseListener	Defines five methods to recognize when the mouse is clicked, enters a component, exits a component, is pressed, or is released.
MouseMotionListener	Defines two methods to recognize when the mouse is dragged or moved.
MouseWheelListener	Defines one method to recognize when the mouse wheel is moved.
TextListener	Defines one method to recognize when a text value changes.
WindowFocusListener	Defines two methods to recognize when a window gains or loses input focus.
WindowListener	Defines seven methods to recognize when a window is activated, closed, deactivated, deiconified, iconified, opened, or quit.

Listener Interface	Handles Event Type	Main Method(s)
ActionListener	Button/menu actions	actionPerformed()
ItemListener	Checkbox/choice selections	itemStateChanged()
KeyListener	Keyboard activity	keyPressed(), keyReleased(), keyTyped()
MouseListener	Mouse clicks and enter/exit	mouseClicked(), etc.
MouseMotionListener	Mouse movement and dragging	mouseMoved(), mouseDragged()
AdjustmentListener	Scrollbar changes	adjustmentValueChanged()
WindowListener	Window events	windowOpened(), windowClosing(), etc.
FocusListener	Focus gained/lost	focusGained(), focusLost()
TextListener	Text value changes	textValueChanged()

1. The ActionListener Interface: This interface defines the actionPerformed() method that is invoked when an action event occurs. Its general form is shown here:

```
void actionPerformed(ActionEvent ae)
```

2. The AdjustmentListener Interface: This interface defines the adjustmentValueChanged() method that is invoked when an adjustment event occurs. Its general form is shown here:

```
void adjustmentValueChanged(AdjustmentEvent ae)
```

3. The ComponentListener Interface This interface defines four methods that are invoked when a component is resized, moved, shown, or hidden. Their general forms are shown here:

```
void componentResized(ComponentEvent ce)
```

```
void componentMoved(ComponentEvent ce)
```

```
void componentShown(ComponentEvent ce)
```

```
void componentHidden(ComponentEvent ce)
```

4. The ContainerListener Interface This interface contains two methods. When a component is added to a container, componentAdded() is invoked. When a component is removed from a container, componentRemoved() is invoked. Their general forms are shown here:

```
void componentAdded(ContainerEvent ce)
```

```
void componentRemoved(ContainerEvent ce)
```

5. The FocusListener Interface This interface defines two methods. When a component obtains keyboard focus, `focusGained( )` is invoked. When a component loses keyboard focus, `focusLost( )` is called.

Their general forms are shown here:

```
void focusGained(FocusEvent fe)
```

```
void focusLost(FocusEvent fe)
```

6. The ItemListener Interface This interface defines the `itemStateChanged( )` method that is invoked when the state of an item changes. Its general form is shown here:

```
void itemStateChanged(ItemEvent ie)
```

7. The KeyListener Interface This interface defines three methods.

The `keyPressed( )` and `keyReleased( )` methods are invoked when a key is pressed and released, respectively. The `keyTyped( )` method is invoked when a character has been entered.

For example, if a user presses and releases the A key, three events are generated in sequence: key pressed, typed, and released. If a user presses and releases the HOME key, two key events are generated in sequence: key pressed and released. The general forms of these methods are shown here: `void keyPressed(KeyEvent ke)` `void keyReleased(KeyEvent ke)` `void keyTyped(KeyEvent ke)`

8. The MouseListener Interface This interface defines five methods.

If the mouse is pressed and released at the same point, mouseClicked() is invoked.

When the mouse enters a component, the mouseEntered() method is called. When it leaves, mouseExited() is called.

The mousePressed() and mouseReleased() methods are invoked when the mouse is pressed and released, respectively.

The general forms of these methods are shown here:

```
void mouseClicked(MouseEvent me)
void mouseEntered(MouseEvent me)
void mouseExited(MouseEvent me)
void mousePressed(MouseEvent me)
void mouseReleased(MouseEvent me)
```

9. The MouseMotionListener Interface: This interface defines two methods.

The mouseDragged() method is called multiple times as the mouse is dragged.

The mouseMoved() method is called multiple times as the mouse is moved.

Their general forms are shown here:

```
void mouseDragged(MouseEvent me)
void mouseMoved(MouseEvent me)
```

10. The MouseWheelListener Interface: This interface defines the mouseWheelMoved() method that is invoked when the mouse wheel is moved. Its

general form is shown here:

```
void mouseWheelMoved(MouseWheelEvent mwe)
```

11. The TextListener Interface: This interface defines the textChanged() method that is invoked when a change occurs in a text area or text field.

Its general form is shown here:

```
void textChanged(TextEvent te)
```

12. The WindowFocusListener Interface: This interface defines two methods: windowGainedFocus() and windowLostFocus().

These are called when a window gains or loses input focus.

Their general forms are shown here:

```
void windowGainedFocus(WindowEvent we)
```

```
void windowLostFocus(WindowEvent we)
```

13. The WindowListener Interface This interface defines seven methods.

The `windowActivated( )` and `windowDeactivated( )` methods are invoked when a window is activated or deactivated, respectively.

If a window is iconified, the `windowIconified( )` method is called.

When a window is deiconified, the `windowDeiconified( )` method is called.

When a window is opened or closed, the `windowOpened( )` or `windowClosed( )` methods are called, respectively.

The `windowClosing( )` method is called when a window is being closed.

The general forms of these methods are:

```
void windowActivated(WindowEvent we)
```

```
void windowClosed(WindowEvent we)
```

```
void windowClosing(WindowEvent we)
```

```
void windowDeactivated(WindowEvent we)
```

```
void windowDeiconified(WindowEvent we)
```

```
void windowIconified(WindowEvent we)
```

```
void windowOpened(WindowEvent we)
```


Adapter classes

An adapter class provides an empty implementation of all methods in an event listener interface. Adapter classes are useful when you want to receive and process only some of the events that are handled by a particular event listener interface.

For example, the **MouseMotionAdapter** class has two methods, **mouseDragged()** and **mouseMoved()**, which are the methods defined by the `MouseMotionListener` interface.

If you were interested in only mouse drag events, then you could simply extend `MouseMotionAdapter` and override **mouseDragged()**.

The empty implementation of **mouseMoved()** would handle the mouse motion events for you.

```
// Demonstrate
import java.applet.Applet;
import java.awt.*;
import java.awt.event.*;

// Main Applet class
public class AdapterDemo extends Applet {
    String msg = "";
    int x = 0, y = 0;

    public void init() {
        // Register MouseAdapter
        MyMouseAdapter mouseAdapter = new MyMouseAdapter(this);
        addMouseListener(mouseAdapter);

        // Register MouseMotionAdapter
        MyMouseMotionAdapter motionAdapter = new MyMouseMotionAdapter(this);
        addMouseMotionListener(motionAdapter);
    }

    public void paint(Graphics g) {
        g.drawString(msg, x, y);
    }
}
```

// Class extending MouseAdapter to handle only mouseClicked

```
class MyMouseAdapter extends MouseAdapter {
```

```
    AdapterDemo adapterDemo;
```

```
    MyMouseAdapter(AdapterDemo demo) {
```

```
        this.adapterDemo = demo;
```

```
    }
```

```
    public void mouseClicked(MouseEvent me) {
```

```
        adapterDemo.msg = "Mouse Clicked";
```

```
        adapterDemo.x = me.getX();
```

```
        adapterDemo.y = me.getY();
```

```
        adapterDemo.repaint();
```

```
    }
```

```
}
```

// Class extending MouseMotionAdapter to handle only mouseDragged

```
class MyMouseMotionAdapter extends MouseMotionAdapter {
```

```
    AdapterDemo adapterDemo;
```

```
MyMouseMotionAdapter(AdapterDemo demo) {  
    this.adapterDemo = demo;  
}  
  
public void mouseDragged(MouseEvent me) {  
    adapterDemo.msg = "Mouse Dragged at (" + me.getX() + ", " + me.getY() + ")";  
    adapterDemo.x = me.getX();  
    adapterDemo.y = me.getY();  
    adapterDemo.repaint();  
}  
}
```

Inner classes

An inner class extending **MouseAdapter** in an Applet is a class defined within the body of the main Applet class that inherits from the **MouseAdapter** class. This inner class is used to handle specific mouse events, such as **mousePressed()**, without needing to implement all methods from the **MouseListener** interface.

Since the inner class is nested within the Applet class, it has direct access to all variables and methods of the enclosing Applet — including methods like **showStatus()**. This allows for simpler and more organized event handling in GUI applications.

```
//Demonstrate
import java.applet.Applet;
import java.awt.event.*;
import java.awt.*;

/*
<applet code="InnerClassDemo" width=400 height=300>
</applet>
*/

public class InnerClassDemo extends Applet {

    public void init() {
        addMouseListener(new MyMouseAdapter()); // Register
inner class for mouse events
    }
}
```

```
// Inner class extending MouseAdapter
class MyMouseAdapter extends MouseAdapter {
    public void mousePressed(MouseEvent me) {
        showStatus("Mouse Pressed at (" + me.getX() + ", " + me.getY() + ")");
    }
}
}
```

Here, **InnerClassDemo** is a top-level class that extends Applet. MyMouseAdapter is an inner class that extends **MouseAdapter**.

➤ Because MyMouseAdapter is defined within the scope of **InnerClassDemo**, it has access to all of the variables and methods within the scope of that class. Therefore, the **mousePressed()** method can call the **showStatus()** method directly.